os

OpenSave 2.0

The Go rewrite — project summary

Steam Cloud for every game you own.

OpenSave automatically syncs your game saves between devices — peer-to-peer, with no Steam, no accounts, and no subscriptions. Version 2.0 is a complete ground-up rewrite of the original Node.js / Electron application in **Go**: one small native binary that keeps every feature, fixes the bugs, and still talks to the old app.

What OpenSave does

- **Auto-detects** saves from Steam, emulators (RetroArch, Dolphin, Ryujinx, Yuzu, PCSX2, RPCS3, and more), repacks, Epic, GOG, and Unreal Engine games — with cover art pulled automatically.
- **Tracks any folder or file**, watched live; only the changed 64 KB blocks ever transfer (SHA-256 delta sync).
- **Syncs peer-to-peer** over local Wi-Fi (zero-config discovery) or across the internet via a relay room code — no port forwarding.
- **Versioned snapshots & branches** — roll back a whole save or a single file; keep parallel playthroughs safe.
- **Smart conflict handling** — when two devices diverge, keep yours, keep theirs, or keep both on a new branch.
- **Optional cloud backup** to Google Drive, Dropbox, OneDrive, WebDAV, a webhook, or a local / NAS folder.
- **Privacy-first** — no accounts, no telemetry; the relay only routes encrypted traffic and never stores your saves.

The headline change: from Electron to a native binary

The original shipped a Node.js daemon plus an Electron desktop shell — a ~200 MB Chromium bundle with a separate Node runtime. The rewrite compiles to a **single ~18 MB native executable** with the sync daemon embedded directly behind a lightweight system WebView. No Node, no Chromium, no install required to run it.

Aspect	OpenSave 1.x (previous)	OpenSave 2.0 (this rewrite)
Language	JavaScript (Node.js)	Go
Desktop shell	Electron — bundled Chromium (~200 MB)	Wails v2 — system WebView (~18 MB)
Runtime needed	Node.js must be present	None — self-contained binary
Storage	Single JSON file, fully rewritten on every change	SQLite — indexed, transactional
File watching	chokidar	fsnotify, safe-write & lock aware
Snapshots	adm-zip	Native ZIP + branches + retention
Processes	Daemon + Electron UI (two processes)	One process (also runs headless)
Tests	JS test scripts	Full suite + 2-daemon E2E, race-checked

Everything above changed **under the hood** — but the REST/WebSocket API and the peer-to-peer wire protocol were deliberately kept identical, so a device running the new Go app pairs and syncs with a device still running the old JavaScript app.

How it was built

The rewrite was delivered in five phases, each fully tested before moving on:

1 · Core daemon	SQLite storage with one-time migration from the old JSON database (games, snapshots, pairings, and cloud tokens all carried over), delta engine, snapshots, file watcher, save auto-detection, and a command-line tool.
2 · LAN sync	UDP device discovery, the pairing handshake with anti-spoofing, and the sync engine — including the lineage-based conflict logic that decides direction without trusting clocks.
3 · Internet & cloud	The stateless relay server (room broker + OAuth proxy), the WAN client that tunnels sync through it, and all six cloud backup providers with PKCE OAuth.
4 · Desktop app	The Wails + Svelte interface in the near-black “Hydra launcher” style requested — sidebar library, snapshot timeline, device pairing, cloud setup, and a live activity log.
5 · System & packaging	System tray with minimize-to-tray, start-on-boot, UPnP port forwarding, the Steam Deck (Decky Loader) plugin carried over unchanged, and a release pipeline for Windows / Linux installers.

Proven, not just built

Rather than trusting the build, the app was run for real and driven end-to-end. That process caught and fixed several genuine bugs:

- A frozen **Activity screen** — an accidental infinite reactive loop in the UI, reproduced through the browser engine and fixed.
- The app couldn't reach its own daemon — a WebView origin / address issue, fixed so live data loads.
- **Fresh clones wouldn't compile** and pushes weren't running tests — both fixed in the project setup and continuous-integration config.
- **Cross-version sync crashed**: the old app sends fractional timestamps the new one rejected. Fixed, then verified by running the real Node app and the Go app side by side — a save edited on one appeared byte-perfect on the other, both directions.

The full automated test suite — unit tests plus an end-to-end harness that spins up two complete daemons and syncs between them — passes with Go's race detector enabled.

Current status

Complete & verified on this machine	Full feature parity · single-binary desktop app · SQLite migration · LAN + relay sync engine · six cloud providers · tray / autostart / UPnP · live interoperability with the original JavaScript app · pushed to the **remake** branch.
Still needs real-world hardware	A true two-machine test across separate networks · the three OAuth providers with live accounts · a relay deployed to the cloud · Steam Deck Game Mode — none of which can be exercised from a single development PC.

In one line: OpenSave 2.0 does everything the original did — same features, same network protocol — but ships as one small, fast native program instead of a heavyweight Electron bundle, with the old version's bugs fixed and its sync engine rebuilt on solid, tested foundations.